



Assignment 4

Vision Transformer for Image Captioning

Objective

In this assignment, you will build an end-to-end **image captioning system** using the **Flickr8k** dataset. You will:

- Use a **pre-trained Vision Transformer (ViT)** as an **image encoder**.
- Extract **patch token embeddings** from the ViT model.
- Feed these embeddings into a **Transformer Decoder** that generates textual captions.
- Train the model to generate captions for unseen images.
- Visualize **attention maps** from the ViT encoder.

Dataset

We will use the **Flickr8k dataset**:

- The dataset is divided into three subsets: Training Set Contains 6000 images, Validation Set: Consists of 1000 images, and Test Set: Includes 1000 images.

You may download it from: <https://huggingface.co/datasets/jxie/flickr8k>

Task Requirements

You must complete the following tasks in order.

1. Data Preparation (10 points)

- Load the Flickr8k images and captions.
- Preprocess captions:
 - lowercase, remove punctuation
 - tokenize
 - build a vocabulary (set a max vocab size, e.g., 10k words)
 - add special tokens: <SOS>, <EOS>, <PAD>, and <UNK>.



2. ViT Encoder Feature Extraction (20 points)

- Load a pre-trained ViT model (e.g., `google/vit-base-patch16-224`).
- Preprocess images to 224×224 .
- Extract encoder outputs (**try both strategies**):
 - **Strategy A –Patch tokens (recommended):**
 - * Exclude CLS token: use only patch tokens as encoder memory.
 - * Output shape per image: `[196, 768]`.
 - * Project these patch tokens with a linear layer to the decoder dimension: `proj: 768 -> d_model`.
 - * Advantages: preserves spatial information, enables fine-grained cross-attention and visualizations.
 - **Strategy B –CLS token (baseline):**
 - * Use only the CLS token as a global image embedding (exclude patch tokens).
 - * Output shape per image: `[1, 768]` (CLS only).
 - * Project CLS to decoder dimension with a linear layer.
 - * Advantages: low memory and compute; simpler baseline to implement and debug.
 - **Evaluation/Comparison requirements (mandatory):**
 - * Implement both strategies and train the decoder with the ViT encoder **frozen** at first for each strategy.
 - * Report quantitative results (BLEU scores) for both strategies on the test split.
 - * Provide qualitative comparisons: show at least 5 examples where patch-token strategy improves grounding or produces more detailed captions than CLS-only, and discuss failure cases.
 - * For attention visualization: use the patch-token strategy to produce **heatmaps** (CLS-only will not produce meaningful spatial maps).
- Add a **linear projection layer** to map ViT embedding dimension (768) to decoder dimension (e.g., 512). to map ViT embedding dimension (768) to decoder dimension (e.g., 512).
- Keep ViT **frozen during initial training**. You may fine-tune later.

3. Transformer Decoder (25 points)

Implement a Transformer decoder to generate captions.



- Input tokens: caption words shifted right (**teacher forcing**).
- Token embedding + positional encoding.
- Decoder layers perform:
 - Masked self-attention over previously generated words.
 - Cross-attention over ViT patch embeddings (*memory*).
- Output a probability distribution over the vocabulary for each time step.
- Implement inference strategies:
 - Greedy decoding *or* Beam search

4. Model Training and Evaluation (20 points)

- Train the decoder using Cross-Entropy Loss.
- Use teacher forcing during training.
- Evaluate on the test split using:
BLEU-1, BLEU-2, BLEU-3, BLEU-4.
- Report sample generated captions.

5. Attention Visualization (25 points)

- Extract **self-attention maps** from the last ViT encoder layers.
- Visualize CLS → Patch attention or Patch → Patch attention.
- Reshape patch attention maps into a 14×14 grid and overlay on the image.
- Provide at least 5 examples with:
 - The original image
 - The generated caption
 - The attention heatmap visualization

Deliverables

- A Jupyter Notebook (.ipynb) file with all code, visualizations, and explanations.
- A short PDF report summarizing your findings, results, and plots.
- Clearly label each section in the notebook corresponding to the tasks above.



Submission Guidelines

- You are allowed to work in teams of up to 3.
- Submit both your **Jupyter Notebook (.ipynb)** and a **PDF report**.
- Each section must be clearly labeled and include **comments** explaining the code.
- Include clean visualizations and markdown explanations.
- Any cases of cheating will result in a score of 0% for all involved teams.

Good Luck!